



PADR

Malo Goossens

Dr. Laurent Philippe, M. Adrien Huygens

Master 1 en Architecture des Systèmes Informatiques
Année académique 2025-2026

Table des matières

1 Analyse du contexte **4**

1.1 Introduction au protocole DNS 4

1.2 Objectif du projet 4

1.3 Périmètre et limites du système 4

1.4 La contrainte de python 4

1.5 Choix de la bibliothèque standard 4

1.6 L'interopérabilité et le respect de la RFC 1035 5

2 Analyse fonctionnelle **6**

2.1 Vision globale du système 6

2.2 User stories 7

2.2.1 Épic 1 : L'Outil Client (CLI) 8

2.2.2 Épic 2 : Le Serveur Récursif (Le Chef d'orchestre) 9

2.2.3 Épic 3 : Le Serveur d'Autorité (La Base de données) 10

2.2.4 Épic 4 : Tolérance aux Pannes & Réplication (Master / Slave) 10

2.3 Épic 5 : Environnement, Qualité et Méthodologie (En tant que Dev) 11

3 Analyse technique **13**

3.1 La Forge Binaire DNS (La Sérialisation) 13

3.1.1 Encodage et masques bit-à-bit 13

3.1.2 Compression des noms et gestion de la section ADDITIONAL 13

3.1.3 Encodage des enregistrements TXT 13

3.2 Stratégie Réseau (UDP vs TCP) 13

3.2.1 Gestion du Fallback TCP 13

3.2.2 Architecture réseau et allocation des ports 13

3.3 Gestion de la Donnée et du Cache 13

3.3.1 Stockage en RAM vs Format BIND 13

3.3.2 Comportement du Cache Manager 13

3.4 La Réplication Master / Slave et Rechargement 13
3.4.1 Mécanismes AXFR et NOTIFY 13
3.4.2 Rechargement à chaud et surveillance de fichier 13
3.5 Architecture Orientée Objet et Moteur Itératif 13
3.5.1 Conception des classes 13
3.5.2 Analyse de la condition d'arrêt itérative 13

4 Implémentation **13**
4.1 Arborescence du Projet 13
4.2 Qualité et Tests (TDD) 13
4.3 Déploiement sous Docker 13
4.4 Refactoring et Retours d'Expérience de Terrain 14
4.4.1 Capsules typées et gestion de la mémoire 14
4.4.2 Bugs critiques identifiés et résolus 14

5 Analyse retrospective **14**
5.1 Bilan des résultats 14
5.2 Limites et perspectives d'améliorations 14

Bibliographie

Introduction

1 Analyse du contexte

1.1 Introduction au protocole DNS

1.2 Objectif du projet

1.3 Périmètre et limites du système

Le périmètre de cette implémentation inclut la résolution itérative des enregistrements A (ipv4) et AAAA (ipv6), la gestion d'un cache local, le fallback vers TCP pour les réponses volumineuses, ainsi que la réplication des zones via AXFR et NOTIFY. Le système est conçu pour être interopérable avec les outils existants tels que dig et nslookup.

Le cahier des charges n'inclut donc pas la mise en œuvre de signatures DNSSEC, le chiffrement, le support des requêtes en IPv6 (même si les enregistrements AAAA sont supportés), ni l'extention EDNS0 permettant de négocier des tailles de trames supérieures à 512 octets en UDP.

L'architecture du système doit cependant faciliter l'ajout éventuel de ces fonctionnalités.

1.4 La contrainte de python

Le language de programmation Python est imposé par les consignes du projet pour la réalisation du programme. Python est un langage de haut niveau, dynamiquement typé, interprété et multiparadigmes, il est apprécié pour sa lisibilité et sa simplicité. Dans le cadre de ce projet, la présence des mécanismes orientés objet, de la gestion automatique de la mémoire et d'une bibliothèque standard riche (`struct`, `socket`, `threading`, `json`) sont d'une utilité considérable. Cependant, l'envoi de messages binaires et la gestion de sockets devant être implémentés manuellement, l'abstraction offerte par Python parfois devenir une gêne. Contrairement à des langages de plus bas niveau (comme le C++) qui permettent d'utiliser des structures avec des pointeurs pour lire les données binaires directement, Python demande un travail constant et explicite de sérialisation et désérialisation. Heureusement, le module `struct` permet de faire le pont entre les objets de haut niveau manipulés par le code et les trames binaires brutes exigées par le réseau.

1.5 Choix de la bibliothèque standard

Dans l'intérêt pédagogique du projet, il est interdit de déléguer l'implémentation de certaines parties du système à des bibliothèques tierces. Afin de ne pas risquer de manquer à cette contrainte, l'ensemble du projet a été développé en utilisant exclusivement la bibliothèque standard Python.

Hors du respect de la contrainte académique, ce choix présente de multiples bénéfices :

— **Portabilité et absence de dépendances :** Le projet peut être exécuté sur n'importe quelle machine disposant d'une installation Python standard, sans nécessiter l'installation de modules supplémentaires.

— **Pérennité et stabilité à long terme :** Les bibliothèques tierces disponibles sur PyPi évoluent rapidement et peuvent déprécier certaines fonctions ou être complètement abandonnées ce qui crée une dette technique. À l'inverse, un projet basé sur la bibliothèque standard Python restera maintenable et fonctionnel sur le long terme.

— **Légèreté et optimisation :** Les bibliothèques externes introduisent des fonctions superflues et des dépendances inutiles. En se limitant à la bibliothèque standard, nous avons la garantie que le code n'inclut que le nécessaire.

— **Sécurité et maîtrise de la chaîne logistique :** Dans le contexte de la cybersécurité moderne, dépendre de multiples paquets externes expose le projet à des failles de type Supply Chain Attack. Restreindre le projet à la bibliothèque standard garantit que la surface d'attaque est limitée à Python lui-même.

1.6 L'interopérabilité et le respect de la RFC 1035

Afin de garantir l'interopérabilité avec les outils existants, le projet doit respecter le format de trame binaire défini par la RFC 1035. Le respect de cette norme permettra au système de communiquer correctement avec les serveurs DNS existants et d'assurer la compatibilité avec des outils tels que dig ou nslookup.

2 Analyse fonctionnelle

2.1 Vision globale du système

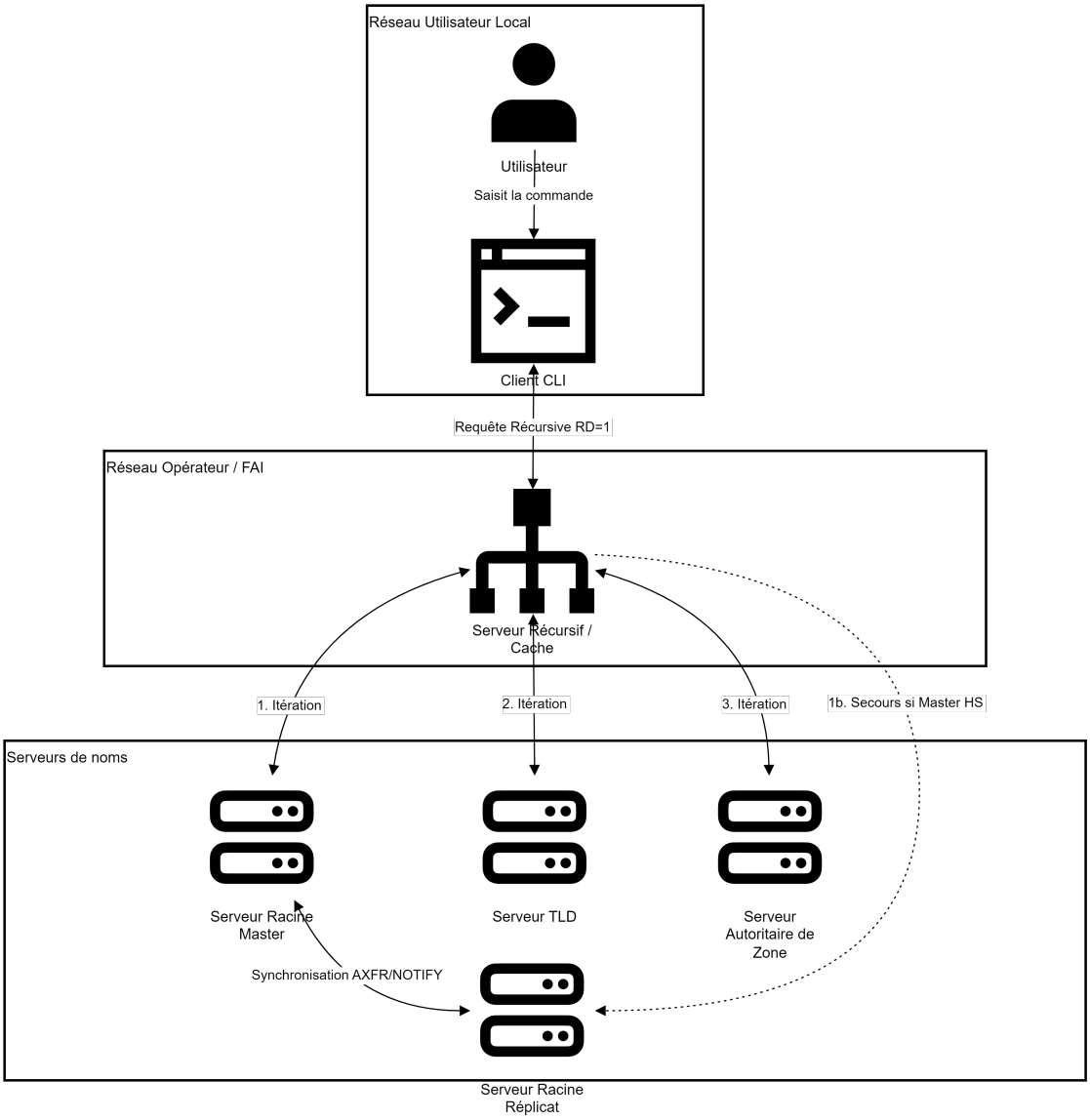


FIGURE 1 – Architecture globale des flux entre le Client, le Résolveur et la Hiérarchie DNS.

Le processus de résolution d'un nom de domaine s'articule autour des trois niveaux d'infrastructure présentés dans le schéma :

1. **Émission de la requête initiale (Réseau Utilisateur Local) :** L'utilisateur saisit une commande réseau (ex : `dig` ou le client Python). Le `Client CLI` génère une trame DNS en

activant le bit `RD=1` (*Recursion Desired*), signalant au résolveur qu'il souhaite obtenir une réponse finale complète sans effectuer lui-même les étapes itératives.

2. **Prise en charge par le résolveur (Réseau Opérateur / FAI) :** La requête parvient au `Serveur Récursif / Cache`. Ce dernier interroge en premier lieu son cache local :
   - **En cas de succès (*Cache Hit*) :** La réponse est directement retournée au client avec le bit `AA=0` et le TTL décrémenté.
   - **En cas d'échec (*Cache Miss*) :** Le résolveur initie la chaîne de résolution itérative vers la hiérarchie DNS.

3. **Cycle de résolution itérative (Serveurs de Noms) :**
   - **1. Première itération (Racine) :** Le résolveur interroge le `Serveur Racine Master` (ou bascule vers le serveur de secours répliqué si le master est hors service, selon la route *1b. Secours si Master HS*). Le serveur racine répond par une délégation vers les serveurs de noms du TLD concerné.
   - **2. Deuxième itération (TLD) :** Le résolveur contacte le `Serveur TLD` (ex : `.be`), qui lui retourne l'adresse IP et les *Glue Records* du serveur autoritaire responsable du domaine.
   - **3. Troisième itération (Autorité) :** Le résolveur interroge le `Serveur Autoritaire de Zone`. Ce dernier consulte sa zone locale et renvoie la réponse autoritaire finale (`AA=1`) contenant les enregistrements demandés (ex : enregistrement `A`).

4. **Mécanisme de réplication sous-jacent :** En arrière-plan, l'intégrité des données entre les serveurs primaires et secondaires est maintenue via le mécanisme de réplication Master/Slave basculant sur les protocoles `NOTIFY` (signalement de modification) et `AXFR` (transfert de zone complet).
5. **Finalisation et mise en cache :** Une fois la réponse enregistrée auprès du serveur d'autorité, le résolveur récursif met en cache le résultat pour la durée spécifiée par le TTL, puis transmet la trame de réponse finale au `Client CLI`.

2.2 User stories

L'aboutissement de l'analyse fonctionnelle est de dresser la liste des *User Stories* (US) et des *Épics* correspondants, qui décrivent les fonctionnalités attendues du système du point de vue de l'utilisateur final.

2.2.1 Épic 1 : L'Outil Client (CLI)

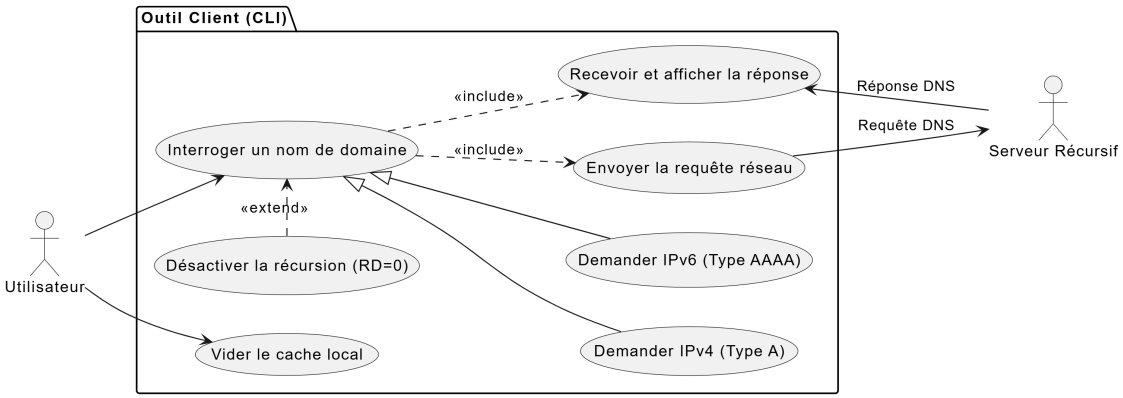


FIGURE 2 – Diagramme de cas d'utilisation du Client CLI.

— **US-1.1 :** En tant qu'utilisateur, je veux pouvoir tester toutes les fonctionnalités du système depuis un client CLI afin de pouvoir vérifier son bon fonctionnement.

— **US-1.2 :** En tant qu'utilisateur, je veux pouvoir résoudre un nom de domaine en adresse IPv4 (Type A) afin de pouvoir me connecter au serveur cible.

— **US-1.3 :** En tant qu'utilisateur, je veux pouvoir résoudre un nom de domaine en adresse IPv6 (Type AAAA) afin de pouvoir me connecter au serveur cible sur un réseau moderne.

— **US-1.4 :** En tant qu'utilisateur, je veux pouvoir envoyer un signal d'administration au serveur récursif afin de vider son cache à distance pour du diagnostic.

— **US-1.5 :** En tant qu'utilisateur, je veux pouvoir désactiver la récursion (‘RD=0‘) via une option du CLI afin d'interroger un serveur de manière ciblée pour du diagnostic.

— **US-1.6 :** En tant qu'utilisateur, je veux pouvoir utiliser la solution sur mon ordinateur personnel en me limitant à de la configuration réseau pour que mon ordinateur effectue les résolutions de façon transparente.

2.2.2 Épic 2 : Le Serveur Récursif (Le Chef d'orchestre)

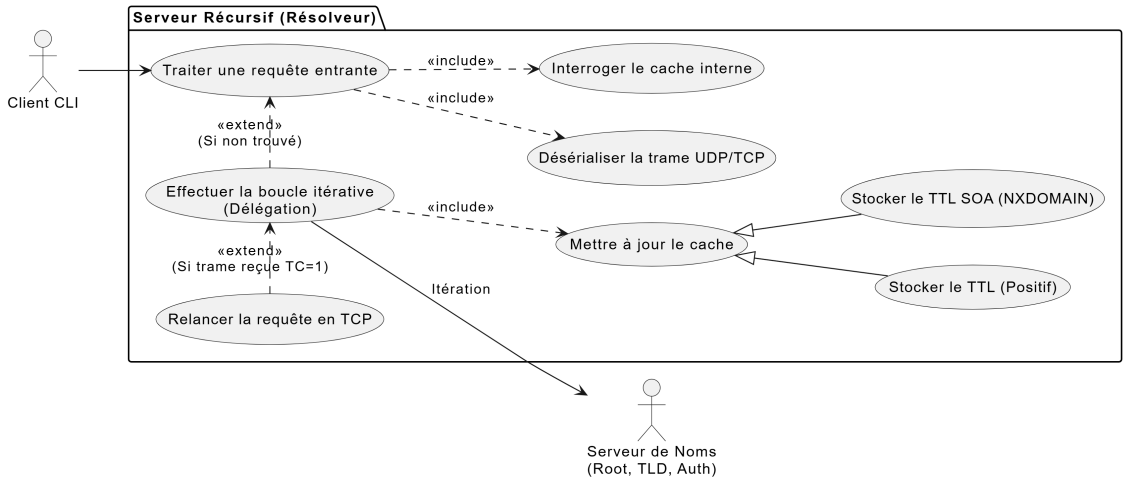


FIGURE 3 – Diagramme de cas d'utilisation du Résolveur Récursif.

— **US-2.1 :** En tant que résolveur, je veux écouter sur le port UDP 53 et désérialiser la trame binaire entrante afin d'extraire le nom de domaine et le type de requête demandés.

— **US-2.2 :** En tant que résolveur, je veux interroger mon cache interne avant d'émettre une requête sur le réseau afin d'accélérer drastiquement le temps de réponse.

— **US-2.3 :** En tant que résolveur, je veux implémenter la boucle de délégation (Root -> TLD -> Autorité) afin de trouver la réponse à la requête.

— **US-2.4 :** En tant que résolveur, je veux extraire le TTL des réponses positives (A/AAAA) et les stocker en mémoire afin de construire mon cache.

— **US-2.5 :** En tant que résolveur, je veux lire l'enregistrement SOA lors d'une erreur *NX-DOMAIN* afin de mettre cette absence en cache (Negative Caching).

— **US-2.6 :** En tant que résolveur, si une réponse UDP contient le flag 'TC=1' (Tronqué), je veux relancer automatiquement ma requête via TCP afin d'obtenir la donnée complète.

— **US-2.7 :** En tant que résolveur, je veux intercepter les messages d'administration du client afin de vider instantanément ma structure de données de cache en RAM.

2.2.3 Épic 3 : Le Serveur d'Autorité (La Base de données)

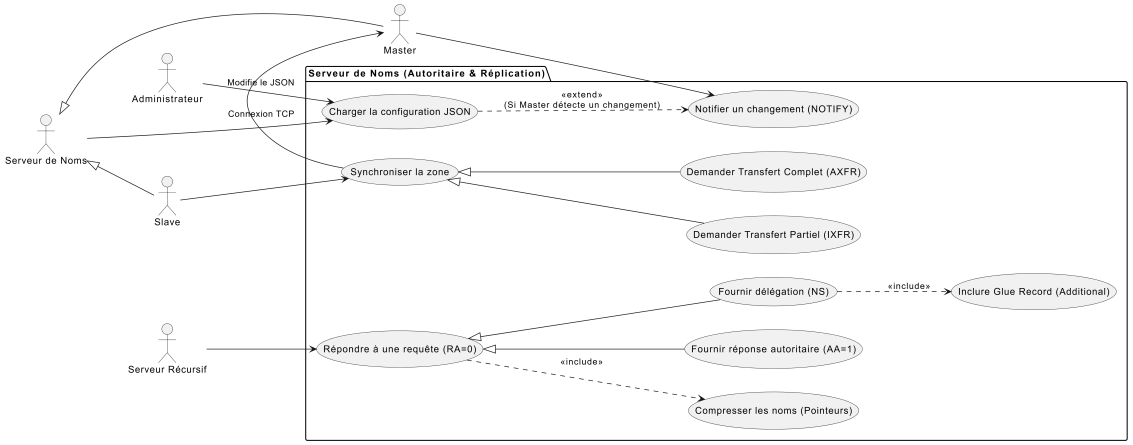


FIGURE 4 – Diagramme de cas d'utilisation du Serveur de Nom.

— **US-3.1 :** En tant que serveur de nom, je veux charger ma base de données (Zone) depuis un fichier JSON au démarrage afin de connaître les enregistrements sous ma responsabilité.

— **US-3.2 :** En tant que serveur de nom, si la requête correspond à une IP que je connais, je veux renvoyer une réponse autoritaire (‘AA=1‘) contenant la ou les IP (RRSets) dans la section *Answer*.

— **US-3.3 :** En tant que serveur de nom, si je ne gère qu'une portion du domaine (ex : Root ou TLD), je veux renvoyer une délégation (enregistrement NS) dans la section *Authority* afin que le serveur récursif puisse continuer sa recherche.

— **US-3.4 :** En tant que serveur de nom, lors d'une délégation, je veux inclure l'adresse IP du serveur délégué dans la section *Additional* (Glue Record) afin de faire gagner du temps au résolveur.

— **US-3.5 :** En tant que serveur de nom, je veux implémenter l'algorithme de compression (pointeur ‘11xxxxxx‘) lors de la création de ma réponse binaire afin de ne pas saturer la limite de 512 octets de l'UDP.

— **US-3.6 :** En tant qu'administrateur, je veux pouvoir déclencher le rechargement du fichier JSON "à chaud" (sans redémarrer le processus du serveur) afin d'ajouter ou modifier des enregistrements sans provoquer d'interruption de service.

2.2.4 Épic 4 : Tolérance aux Pannes & Réplication (Master / Slave)

— **US-4.1 :** En tant que serveur Racine Réplica (Slave), je veux me connecter en TCP au serveur Racine Master à mon démarrage et demander un transfert de zone complet (AXFR) afin d'obtenir la configuration initiale.

— **US-4.2 :** En tant que serveur Racine Master, je veux accepter les requêtes de type AXFR en

TCP et renvoyer l'intégralité de mon fichier de zone afin d'assurer la redondance requise.

— **US-4.3 :** En tant que serveur Racine Master, lorsque mon fichier JSON est modifié, je veux envoyer un message UDP (OpCode 4, NOTIFY) à mon Slave afin de le prévenir instantanément d'un changement.

— **US-4.4 :** En tant que serveur Racine Réplica (Slave), à la réception d'un NOTIFY ou après l'expiration de mon timer de Refresh, je veux comparer mon numéro de série (SOA) avec celui du Master afin de savoir si je dois me mettre à jour.

— **US-4.5 :** En tant que serveur Racine Réplica, si le numéro de série du Master est plus grand, je veux effectuer une requête de transfert incrémental (IXFR) en TCP pour télécharger uniquement les nouveaux enregistrements.

2.3 Épic 5 : Environnement, Qualité et Méthodologie (En tant que Dev)

— **US-5.1 :** En tant que développeur, je veux qu'un pipeline d'Intégration Continue exécute automatiquement mes tests unitaires à chaque push afin de valider la non-régression de mon code.

3 Analyse technique

3.1 La Forge Binaire DNS (La Sérialisation)

3.1.1 Encodage et masques bit-à-bit

3.1.2 Compression des noms et gestion de la section ADDITIONAL

3.1.3 Encodage des enregistrements TXT

3.2 Stratégie Réseau (UDP vs TCP)

3.2.1 Gestion du Fallback TCP

3.2.2 Architecture réseau et allocation des ports

3.3 Gestion de la Donnée et du Cache

3.3.1 Stockage en RAM vs Format BIND

3.3.2 Comportement du Cache Manager

3.4 La Réplication Master / Slave et Rechargement

3.4.1 Mécanismes AXFR et NOTIFY

3.4.2 Rechargement à chaud et surveillance de fichier

3.5 Architecture Orientée Objet et Moteur Itératif

3.5.1 Conception des classes

3.5.2 Analyse de la condition d'arrêt itérative

4 Implémentation

4.1 Arborescence du Projet

4.2 Qualité et Tests (TDD)

4.3 Déploiement sous Docker

Le projet est déployé dans un environnement *Docker Compose*, permettant de simuler un réseau DNS complet avec des serveurs de noms et un résolveur récursif. De cette manière, il est possible

d’essayer le code en instanciant plusieurs serveurs sur une machine unique.

4.4 Refactoring et Retours d’Expérience de Terrain

4.4.1 Capsules typées et gestion de la mémoire

4.4.2 Bugs critiques identifiés et résolus

5 Analyse retrospective

5.1 Bilan des résultats

5.2 Limites et perspectives d’améliorations

Déclaration d’usage de l’IA générative

Conformément au Guide du Projet Innovant de l’Hénallux, je déclare avoir utilisé le grand modèle linguistique Gemini (Google) en tant qu’assistant à la rédaction lors de ce projet. Son usage s’est limité à la reformulation académique de textes préalablement rédigés par mes soins, à la mise en forme typographique des équations mathématiques (génération de code LaTeX) et à l’aide à la structuration des sections du rapport. L’entièreté de la réflexion, des choix techniques et de l’analyse sont le fruit de mon travail personnel.

Google, "Gemini," May 2024. [Large language model]. Available : https ://gemini.google.com/

Table des figures

1 Architecture globale des flux entre le Client, le Résolveur et la Hiérarchie DNS. . 6

2 Diagramme de cas d'utilisation du Client CLI. 8

3 Diagramme de cas d'utilisation du Résolveur Récursif. 9

4 Diagramme de cas d'utilisation du Serveur de Nom. 10